

Pandas Data Frames

DSE5002, HD Sheets, July 2024, updated 6/26/2025

Pandas is a Python implementation of the R or SQL style data frame or data table

Indexing is a bit different, and there are some other "wrinkles" to indexing

There are a lot of member functions (aka *methods*) in Pandas to do a lot of basic data processing and graphing of data within a data frame

Pandas data frames have variables along columns, which can be of different types

Each row is an individual event (a person, a transaction, an invoice, etc)

The dataframes in Pandas or R mimic closely the *table* seen in SQL databases.

More resources

<https://pandas.pydata.org/>

```
In [1]: import pandas as pd
```

Free WiFi in Boston

we will load a data frame describing public wifi access sites in Boston, it came from the site

`https://data.boston.gov/` *most cities have sites like this one*

the file is called wicked_free_wifi_boston.csv

There is a `read_csv` function in pandas, that will attempt to assign variable types to columns

You will need to insert the full file path into the variable `infile` below, or make sure that the file is in the current working directory

below is the command from the `os` library to show the current working directory

we can use `os.chdir()` to change the current working directory

```
In [2]: import os

        #find out what the current working directory is using the getcwd function from the os library
        os.getcwd()
```

```
Out[2]: '/Users/ramakrishnayalamanchili/Documents/Data Science Documents /module 2 '
```

```
In [3]: # Make sure that the file Wicked_Free_WiFi_Locations.csv
# is in the current working directory by running os.listdir() to see directory c

os.listdir()
```

```
Out[3]: ['.ipynb_checkpoints',
'Wicked_Free_WiFi_Locations-1.csv',
'PairProgramming_Pandas-1.ipynb']
```

```
In [6]: # before running this cell, copy "Wicked_Free_WiFi_Locations.csv" into the curre
# if it is not already there

infile="Wicked_Free_WiFi_Locations-1.csv"

wifi=pd.read_csv(infile)
```

```
In [7]: # head function, called as a methd belonging to the dataframe (a python object)
# wifi is said to be an instance of a python dataframe

wifi.head(7)
```

```
Out[7]:
```

	X	Y	neighborhood_id	neighborhood_name	device_serial	c
0	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q3AK-SUL7-7FC4	
1	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q2ZY-RF99-YN45	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
3	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-DU9C-2UXZ	
4	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-FGAN-3AR9	
5	-7.913037e+06	5.209642e+06	N_568579452955527921	Parks	Q2EK-4PWN-GALS	FRAI
6	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SSY2-PBYW	RC

```
In [8]: #unlike the R data frames, head doesn't show us the data types
# we need to do that manually, looking at the attribute dtypes

wifi.dtypes
```

```
Out[8]: X          float64
Y          float64
neighborhood_id  object
neighborhood_name object
```

```

device_serial      object
device_connectedto object
device_address     object
device_lat         float64
device_long        float64
device_tags        object
etl_updatedtimestamp object
is_current         int64
org1               object
org2               object
inside_outside     object
landmark           object
ObjectId           int64
dtype: object

```

What is the data type "object" in Pandas

An object is a string storage form

```

In [9]: # Generating a Summary
        # describes only numeric values

        wifi.describe()

```

```

Out[9]:

```

	X	Y	device_lat	device_long	is_current	ObjectId
count	2.830000e+02	2.830000e+02	283.000000	283.000000	297.0	297.000000
mean	-7.912135e+06	5.210210e+06	42.327796	-71.075917	1.0	149.000000
std	3.034883e+03	4.447129e+03	0.029537	0.027263	0.0	85.880731
min	-7.922403e+06	5.198613e+06	42.250739	-71.168161	1.0	1.000000
25%	-7.913761e+06	5.207725e+06	42.311295	-71.090520	1.0	75.000000
50%	-7.912078e+06	5.210305e+06	42.328431	-71.075407	1.0	149.000000
75%	-7.910171e+06	5.214371e+06	42.355431	-71.058271	1.0	223.000000
max	-7.904348e+06	5.218989e+06	42.386080	-71.005970	1.0	297.000000

Subsetting and slicing in pandas

Slicing or subsetting refers to the process of extracting specific rows or columns from a dataframe that have the specific types of information we will need.

```

In [10]: # when accessing a column, there are several options

        n_name=wifi.neighborhood_name
        w_address=wifi["device_address"]

```

Pandas series

If we extract a column it is in the form of a pandas data series, which still has a lot of pandas style member functions

```
In [11]: type(n_name)
```

```
Out[11]: pandas.core.series.Series
```

```
In [12]: # we can convert this to a list, using a member function

#a list is a basic data type in python, the pandas.core.series.Series is a impor

# Classes are custom created data storage and manipulation "objects", in the obj
# is coming later

n_name_list=n_name.to_list()
type(n_name_list)
```

```
Out[12]: list
```

```
In [13]: # dimensions of a dataframe are obtained using the attribute shape
print(wifi.shape)
print(n_name.shape)
```

```
(297, 17)
(297,)
```

```
In [14]: #indexing several columns at a time is done by listing the column names we want

wifi[["X", "Y"]].head()
```

```
Out[14]:
```

	X	Y
0	NaN	NaN
1	NaN	NaN
2	-7.912255e+06	5.206228e+06
3	NaN	NaN
4	NaN	NaN

Question/Action

use head to show the first 5 rows of the neighborhood id and name

```
In [16]: wifi.columns
column_names_to_subset = ['neighborhood_id', 'neighborhood_name']
wifi[column_names_to_subset].head(5)
```

Out [16]:

	neighborhood_id	neighborhood_name
0	L_601230550253963849	Mobile WiFi Kit 1
1	L_601230550253963849	Mobile WiFi Kit 1
2	L_601230550253964116	Nubian-Bus-Stop
3	L_601230550253964116	Nubian-Bus-Stop
4	L_601230550253964116	Nubian-Bus-Stop

In [17]: *#Basic calculations using built in *methods* or member functions in Pandas*

```
print(wifi.device_lat.max())
print(wifi.device_lat.min())
print(wifi.device_lat.mean())
```

```
42.38608
42.2507393
42.3277957622368
```

Question/Action

Use google to look up how to calculate the standard deviation of a Pandas variable

Use this to find the standard deviation of the longitude in this data set

In [21]:

```
std_value = wifi.device_long.std()
print(f'Standard Value = {std_value}')
```

```
Standard Value = 0.02726282192230917
```

Slicing

Slicing is the proces of extracting specific roles that meet some criteria or condition.

It is also called filtering

This is a common task when working with dataframes or SQL tables

In [22]:

```
# filtering rows using conditional dependence

#let's find all devices with latitude above 42.3271405

above_wifi=wifi[wifi.device_lat>=42.3271405]
above_wifi.head()
```

Out [22]:

	X	Y	neighborhood_id	neighborhood_name	device_serial	dev
6	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SSY2-PBYW	R

	X	Y	neighborhood_id	neighborhood_name	device_serial	dev
7	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SU8N-5VU8	RO
15	-7.912357e+06	5.210581e+06	N_579275502070532581	Roxbury	Q2CK-HDFV-VYBC	RO
18	-7.912846e+06	5.210317e+06	N_579275502070532581	Roxbury	Q2CK-SF4S-8JL2	I
20	-7.912858e+06	5.210361e+06	N_579275502070532581	Roxbury	Q2CK-MR56-4QY6	I

In [23]: `#slicing by values in a set, notice that pandas has a isin() member function for
wifi[wifi.neighborhood_name.isin(["Parks","Charlestown"])]`

Out [23]:		X	Y	neighborhood_id	neighborhood_name	device_serial	
	5	-7.913037e+06	5.209642e+06	N_568579452955527921	Parks	Q2EK-4PWN-GALS	F
	28	-7.910979e+06	5.214437e+06	N_568579452955527921	Parks	Q3AK-CVAE-CUZU	
	44	-7.916707e+06	5.202882e+06	N_568579452955527921	Parks	Q2AK-U4TT-J95L	
	188	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-V3LS-5V65	
	202	-7.910805e+06	5.214387e+06	N_568579452955527921	Parks	Q3AK-DGSZ-GM7H	
	203	-7.911261e+06	5.214274e+06	N_568579452955527921	Parks	Q3AK-EK6U-T4FW	
	205	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-CWUY-RBGW	
	214	-7.911012e+06	5.214442e+06	N_568579452955527921	Parks	Q3AK-DRLB-LEZF	
	223	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-SNTB-WJWA	

	X	Y	neighborhood_id	neighborhood_name	device_serial
226	-7.911133e+06	5.214283e+06	N_568579452955527921	Parks	Q3AK-CR6K-YPBC
227	-7.916761e+06	5.208413e+06	N_568579452955527921	Parks	Q2CK-7ANU-RF72
228	-7.910954e+06	5.213995e+06	N_568579452955527921	Parks	Q3AK-CR92-A992
229	-7.916761e+06	5.208413e+06	N_568579452955527921	Parks	Q2CD-6YGF-H7PW
230	-7.910787e+06	5.214274e+06	N_568579452955527921	Parks	Q3AK-DGWD-NEFT
231	-7.910746e+06	5.214357e+06	N_568579452955527921	Parks	Q3AK-CVAW-H5UM
240	-7.916761e+06	5.208413e+06	N_568579452955527921	Parks	Q2CK-7CSR-NUQK
241	-7.911216e+06	5.214476e+06	N_568579452955527921	Parks	Q3AK-CRWB-RXVL
242	-7.910801e+06	5.214373e+06	N_568579452955527921	Parks	Q3AK-CQWK-ZYPY

In [24]:

```
# notna returns true or false depending on whether there are Nan values in the list
# the list of True/False values produced by notna can be used to slice, so select
# whose X coordinates are not NA

wifi[wifi.X.notna()].head()
```

Out[24]:

	X	Y	neighborhood_id	neighborhood_name	device_serial	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
5	-7.913037e+06	5.209642e+06	N_568579452955527921	Parks	Q2EK-4PWN-GALS	FRAI
6	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SSY2-PBYW	RC
7	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SU8N-5VU8	RC
8	-7.913466e+06	5.208391e+06	N_579275502070532581	Roxbury	Q2CK-H5VS-5UKS	

	X	Y	neighborhood_id	neighborhood_name	device_serial	c
--	---	---	-----------------	-------------------	---------------	---

In [25]:

```
# there is also a str.contains function, which searches for a regex string within a string
# we will talk about regex in more detail later

# notice the use of .str.contains(), a string function within pandas

# see https://pandas.pydata.org/docs/user_guide/text.html for a bunch of examples

wifi[wifi.neighborhood_name.str.contains("town")]
```

Out[25]:

	X	Y	neighborhood_id	neighborhood_name	device_serial	d
53	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-4FLA- SJCJ	
54	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-HJRQ- D68Q	
55	-7.909698e+06	5.215107e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-ARNB- HFAK	
56	-7.909702e+06	5.215082e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-PMFQ- JUX3	
57	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-3S8D- CZPE	
58	-7.909943e+06	5.215065e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AK-C98F- JUR2	
59	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-TBT5- D72C	
61	-7.909943e+06	5.215065e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AK-D5DU- 9HST	
62	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-ZUF3- Y73P	
69	-7.909890e+06	5.215063e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-F6RV- VVHP	
70	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-ZXW8- H9ZW	
71	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-3A2V- JNVG	

	X	Y	neighborhood_id	neighborhood_name	device_serial	d
152	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-JBPL- DSNV	
153	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-ZDPZ- 2LNE	
154	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-QYJ5- RXEJ	
155	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-2N4M- BWUD	
156	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-TF7U- TX4P	
157	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-95XZ- 766Y	
158	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-M6TA- MFWJ	
159	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-R6BQ- D5KN	
160	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-M9YS- PAT8	
161	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-E75T- LLQX	
162	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-BFV5- YGNC	
163	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-8YNT- BM6Q	
164	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-CANN- 4NB2	
167	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-2F27- DVJ3	
168	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-EETG- KYUS	
169	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-A943- FE9F	

	X	Y	neighborhood_id	neighborhood_name	device_serial	d
170	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-WU87- WVGK	
171	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-JXU4- QPQ3	
172	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-A793- TEM9	
188	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-V3LS- 5V65	
205	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-CWUY- RBGW	
223	-7.910482e+06	5.218089e+06	N_568579452955538062	Charlestown	Q2CK-SNTB- WJWA	
256	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-GDTR- 3C3W	
257	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-MULH- 9V9B	
258	-7.910171e+06	5.215103e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-JLUF- 293V	
275	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-QMTD- 57LD	
278	-7.910171e+06	5.215103e+06	N_601230550253966673	Downtown Boston - City Hall Plaza and Pavilion	Q3AE-Q4LQ- 6HK3	
289	-7.909896e+06	5.215085e+06	N_601230550253961310	Downtown Boston - City Hall - Quincy Market	Q3AE-Q7YG- V974	

In [26]:

```
# Row and column specification
# use paired conctions on [row,column]
# we now have to use the method .loc[] to do this

wifi.loc[wifi.X.notna(),"X"].head()
```

Out[26]:

```
2    -7.912255e+06
5    -7.913037e+06
6    -7.913800e+06
7    -7.913800e+06
```

8 -7.913466e+06
Name: X, dtype: float64

In [27]: *# you have to have a boolean return type in the row indexing function or a set of column names to get several of them*

```
wifi.loc[wifi.neighborhood_name.str.contains("Charlestown"),['device_lat','device_long']]
```

Out[27]:

	device_lat	device_long
188	42.380109	-71.06107
205	42.380109	-71.06107
223	42.380109	-71.06107

In [28]: *# Indexing using integer values is done using the iloc[] function*

so remember- used .loc for boolean and named columns, .iloc for Integer locations

```
wifi.iloc[0:8,0:6]
```

Out[28]:

	X	Y	neighborhood_id	neighborhood_name	device_serial	category
0	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q3AK-SUL7-7FC4	
1	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q2ZY-RF99-YN45	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
3	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-DU9C-2UXZ	
4	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-FGAN-3AR9	
5	-7.913037e+06	5.209642e+06	N_568579452955527921	Parks	Q2EK-4PWN-GALS	FRAI
6	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SSY2-PBYW	RC
7	-7.913800e+06	5.210828e+06	N_579275502070532581	Roxbury	Q2CK-SU8N-5VU8	RC

Plotting with Pandas functions

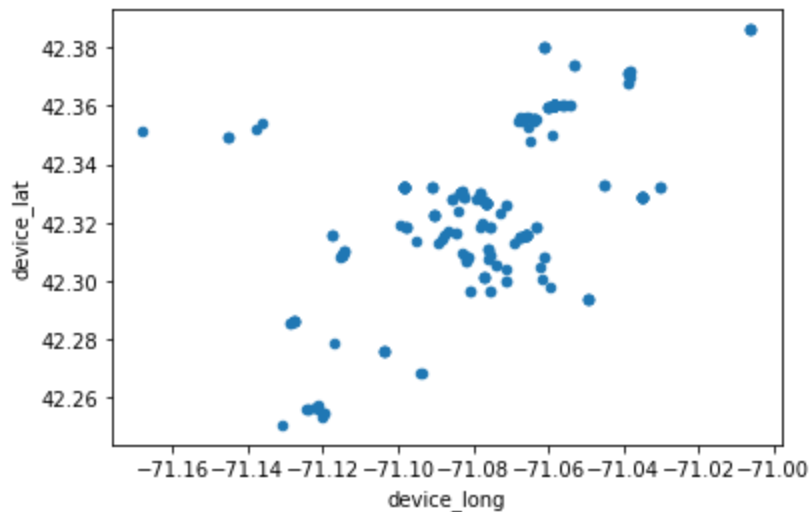
Pandas has basic plotting built in

I typically use Matplotlib, but Pandas has the basics, scatterplots (aka biplots), box and whisker plots, and histograms

```
In [29]: #Pandas built in plots

wifi.plot.scatter(x="device_long",y="device_lat")
```

```
Out[29]: <AxesSubplot:xlabel='device_long', ylabel='device_lat'>
```

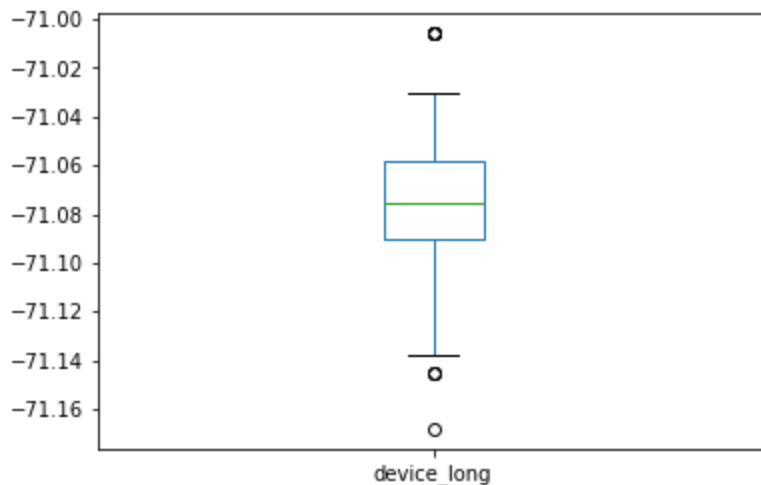


```
In [30]: #here is a boxplot aka a box and whisker plot

# the green bar in the middle of the box is the median value,
# the ends of the box are the 25% and 75% quantiles (so 50% of the data is in th
# the ends of the whiskers are the farthest specimen that is within 1.5 times th
# the box away from the box, this is roughly the typical expected range of value
# points beyond the whiskers are nominally outliers, although this identificatio
# that they are outliers is based on questionable assumptions

wifi[["device_long"]].plot.box()
```

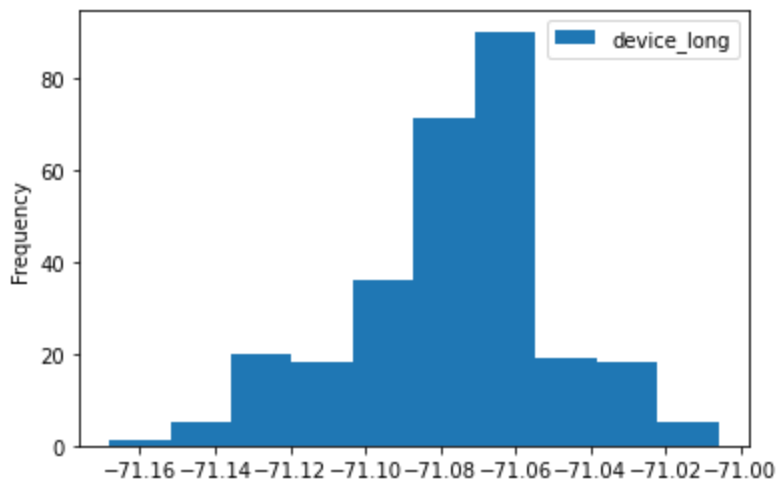
```
Out[30]: <AxesSubplot:>
```



```
In [31]: #histogram

wifi[["device_long"]].plot.hist()
```

Out [31]: <AxesSubplot:ylabel='Frequency'>



```
In [32]: #creating new columns

#just name the column and assign a value

# this is a nonsensical value, but it shows the idea

wifi["x over y"] = wifi.X/wifi.Y

wifi.head()
```

```
Out [32]:
```

	X	Y	neighborhood_id	neighborhood_name	device_serial	device_type
0	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q3AK-SUL7-7FC4	
1	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q2ZY-RF99-YN45	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
3	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-DU9C-2UXZ	F
4	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-FGAN-3AR9	F

Aggregation or grouping for tables and statistics

Pandas as a nice groupby function, reminiscent of the dplyr methods

```
In [33]: # we specify the columns we want to work with in the dataframe, then specify which
# at the end, we add a Pandas summary function

# Note, if we had more grouping variables the input to groupby could be a list
```

```
wifi[["device_long","device_lat","neighborhood_name"]].groupby("neighborhood_name")
```

Out[33]:

	device_long	device_lat
neighborhood_name		
2 Center Plaza	-71.060201	42.359796
Alston Brighton	-71.146410	42.350730
BCYF Indoor	-71.084504	42.320980
BCYF Tremont	-71.098229	42.332227
BCYF-Curley	-71.035150	42.329115
Bolling	-71.083664	42.330141
Charlestown	-71.061070	42.380109
City Hall Truck	NaN	NaN
Dorchester	-71.067166	42.306111
Downtown Boston - City Hall - Quincy Market	-71.057176	42.360229
Downtown Boston - City Hall Plaza and Pavilion	-71.058271	42.360291
East Boston	-71.038550	42.370514
East Boston Senior Center	-71.005970	42.386080
Hyde Park	-71.122637	42.255603
Jamaica Plain	-71.114836	42.309554
Maintenance	-71.063579	42.347038
Mattahunt BCYF	-71.103910	42.275826
Mattapan-Bus-stop	-71.093735	42.268617
Mobile WiFi Kit 1	NaN	NaN
Mobile WiFi Kit 2	NaN	NaN
Mobile WiFi Kit 3	NaN	NaN
Mobile WiFi Kit 4	NaN	NaN
Mobile WiFi Kit 5	NaN	NaN
NOC-TEST	-71.076985	42.331899
Navy Yard	-71.053276	42.373728
Nubian-Bus-Stop	-71.077000	42.301350
OPAT	-71.082958	42.331086
Parks	-71.080528	42.340156
Roslindale	-71.128088	42.286149
Roxbury	-71.082860	42.320404
South Boston	-71.037557	42.332484
Strand Theatre - External	-71.065961	42.315948

	device_long	device_lat
neighborhood_name		
Strand Theatre - Internal	-71.065961	42.315948
YEE Tremont	-71.098240	42.332229

Multiple grouping variables

```
In [34]: # we can use groupby to get counts per grouping variable as well
# I tried using is_current as a grouping variable, but they are all 1, indicating
wifi[["device_long","device_lat","neighborhood_name","is_current"]].groupby(["ne
```

```
Out[34]:
```

		device_long	device_lat
neighborhood_name	is_current		
2 Center Plaza	1	6	6
Alston Brighton	1	6	6
BCYF Indoor	1	27	27
BCYF Tremont	1	9	9
BCYF-Curley	1	12	12
Bolling	1	6	6
Charlestown	1	3	3
City Hall Truck	1	0	0
Dorchester	1	24	24
Downtown Boston - City Hall - Quincy Market	1	16	16
Downtown Boston - City Hall Plaza and Pavilion	1	21	21
East Boston	1	9	9
East Boston Senior Center	1	5	5
Hyde Park	1	12	12
Jamaica Plain	1	6	6
Maintenance	1	6	6
Mattahunt BCYF	1	5	5
Mattapan-Bus-stop	1	2	2
Mobile WiFi Kit 1	1	0	0
Mobile WiFi Kit 2	1	0	0
Mobile WiFi Kit 3	1	0	0
Mobile WiFi Kit 4	1	0	0
Mobile WiFi Kit 5	1	0	0
NOC-TEST	1	2	2

		device_long	device_lat
neighborhood_name	is_current		
Navy Yard	1	3	3
Nubian-Bus-Stop	1	5	5
OPAT	1	2	2
Parks	1	15	15
Roslindale	1	5	5
Roxbury	1	52	52
South Boston	1	6	6
Strand Theatre - External	1	3	3
Strand Theatre - Internal	1	10	10
YEE Tremont	1	5	5

Categorical data

We can set data to be of type Categorical, which is akin to a factor

It is also possible to use integer group codes or dummy coding to represent categories or factors, this is done using utility tools in libraries such as scikit-learn or keras that focus on modeling

```
In [35]: wifi['neighborhood_name']=pd.Categorical(wifi.neighborhood_name)
wifi.head()
```

```
Out[35]:
```

	X	Y	neighborhood_id	neighborhood_name	device_serial	device
0	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q3AK-SUL7-7FC4	
1	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q2ZY-RF99-YN45	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
3	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-DU9C-2UXZ	F
4	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-FGAN-3AR9	F

```
In [36]: # did this work
wifi.dtypes
```



```
Out[36]: X          float64
Y          float64
neighborhood_id      object
neighborhood_name    category
device_serial        object
device_connectedto   object
device_address       object
device_lat           float64
device_long          float64
device_tags          object
etl_updatedtimestamp object
is_current           int64
org1                 object
org2                 object
inside_outside       object
landmark             object
ObjectId            int64
x over y            float64
dtype: object
```

Question/Action

What other variables should be Categorical variables (there aren't many)

Convert this variable to a category

```
In [38]: #inside_outside; landmark
wifi['inside_outside'] = pd.Categorical(wifi.inside_outside)
wifi['landmark'] = pd.Categorical(wifi.landmark)
wifi.head()
```

```
Out[38]:
```

	X	Y	neighborhood_id	neighborhood_name	device_serial	device
0	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q3AK-SUL7-7FC4	
1	NaN	NaN	L_601230550253963849	Mobile WiFi Kit 1	Q2ZY-RF99-YN45	
2	-7.912255e+06	5.206228e+06	L_601230550253964116	Nubian-Bus-Stop	Q3AE-QFTK-E55W	
3	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-DU9C-2UXZ	F
4	NaN	NaN	L_601230550253964116	Nubian-Bus-Stop	Q3AK-FGAN-3AR9	F

Dummy Coding

It looks like Pandas can generate dummy codes for us

Pandas does not have a 'factor' variable, so in models like multiple regression, logistic regression or neural networks, we use dummy coding to code categorical variables. You will see more on this later.

What does this look like?

What does a True in this table seem to mean?

This is also called "one-hot" encoding, since there is only one "True" per row of the table

```
In [40]: pd.get_dummies(wifi.neighborhood_name)
```

```
Out[40]:
```

	2 Center Plaza	Alston Brighton	BCYF Indoor	BCYF Tremont	BCYF- Curley	Bolling	Charlestown	City Hall Truck	Dorchester	Downt Bost City H Quin Mar
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...
292	0	0	0	0	0	0	0	0	0	0
293	0	0	0	0	0	1	0	0	0	0
294	0	0	0	0	0	1	0	0	0	0
295	0	0	0	0	0	1	0	0	0	0
296	0	0	0	0	0	1	0	0	0	0

297 rows × 34 columns

Question/Action

Create a dummy coding for the variable that you turned into a Categorical variable in the Question above

```
In [39]: pd.get_dummies(wifi.landmark)
```

```
Out[39]:
```

	Bolling	Bolling CoB Employee	Boston Common	Boston Latin Academy	Brighton High School	Centre South Main Street Curtis Hall	Centre South Main Street Lightpole	Charlestown	Charlestown Athle Buildi
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0

	Bolling	Bolling CoB Employee	Boston Common	Boston Latin Academy	Brighton High School	Centre South Main Street Curtis Hall	Centre South Main Street Lightpole	Charlestown	Charlestown Athletic Building
3	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	
292	0	0	0	0	0	0	0	0	
293	0	1	0	0	0	0	0	0	
294	0	1	0	0	0	0	0	0	
295	1	0	0	0	0	0	0	0	
296	0	1	0	0	0	0	0	0	

297 rows × 58 columns

date-time values

It looks like we have one date-time variable in the dataset right now

etl_updatedtimestamp

it looks like a fairly standard format

```
In [41]: wifi.etl_updatedtimestamp.head(5)
```

```
Out[41]: 0    2024/08/20 04:31:34+00
1    2024/08/20 04:31:34+00
2    2024/08/20 04:31:38+00
3    2024/08/20 04:31:38+00
4    2024/08/20 04:31:38+00
Name: etl_updatedtimestamp, dtype: object
```

```
In [42]: wifi.etl_updatedtimestamp=pd.to_datetime(wifi.etl_updatedtimestamp)
wifi.etl_updatedtimestamp.head()
```

```
Out[42]: 0    2024-08-20 04:31:34+00:00
1    2024-08-20 04:31:34+00:00
2    2024-08-20 04:31:38+00:00
3    2024-08-20 04:31:38+00:00
4    2024-08-20 04:31:38+00:00
Name: etl_updatedtimestamp, dtype: datetime64[ns, UTC]
```

```
In [43]: # We can now get days, months, years

wifi.etl_updatedtimestamp.dt.day.head()
```

```
Out[43]: 0    20
         1    20
         2    20
         3    20
         4    20
         Name: etl_updatedtimestamp, dtype: int64
```

```
In [44]: wifi.etl_updatedtimestamp.dt.month.head()
```

```
Out[44]: 0     8
         1     8
         2     8
         3     8
         4     8
         Name: etl_updatedtimestamp, dtype: int64
```

```
In [ ]: wifi.etl_updatedtimestamp.dt.year.head()
```

Converting to Long form

uses the melt function

<https://pandas.pydata.org/docs/reference/api/pandas.melt.html>

Form more ideas on wide to long, look up

Pandas pivot pandas pivot_table pandas unstack pandas wide_to_long

```
In [45]: df = pd.DataFrame({'A': {0: 'a', 1: 'b', 2: 'c'},
                           'B': {0: 1, 1: 3, 2: 5},
                           'C': {0: 2, 1: 4, 2: 6}})
```

df

```
Out[45]:
```

	A	B	C
0	a	1	2
1	b	3	4
2	c	5	6

```
In [ ]: pd.melt(df, )
```

```
In [46]: # note we specify a list of i variables and a list of value variables, much lik

pd.melt(df, id_vars=['A'], value_vars=['B', 'C'])
```

```
Out[46]:
```

	A	variable	value
0	a	B	1

	A	variable	value
1	b	B	3
2	c	B	5
3	a	C	2
4	b	C	4
5	c	C	6

```
In [47]: # alternative form, two id variables
# this is a "composite key" form

pd.melt(df, id_vars=['A','C'], value_vars=['B'])
```

```
Out[47]:
```

	A	C	variable	value
0	a	2	B	1
1	b	4	B	3
2	c	6	B	5

```
In [48]: # create df2 for question/action

df2 = pd.DataFrame({'A': {0: 'a', 1: 'b', 2: 'c', 3: 'd'},
                    'B': {0: 1, 1: 3, 2: 5, 3: 6},
                    'C': {0: 2, 1: 4, 2: 6, 3: 8},
                    'D': {0: "Biscuit", 1: "Chips", 2: "Banana", 3: "hard case"}})

df2
```

```
Out[48]:
```

	A	B	C	D
0	a	1	2	Biscuit
1	b	3	4	Chips
2	c	5	6	Banana
3	d	6	8	hard case

Question/Action

Melt df2 to wide form, using D and A as the index variables, assign the other two columns as values

```
In [50]: pd.melt(df2, id_vars=['D','A'], value_vars=['B', 'C'])
```

```
Out[50]:
```

	D	A	variable	value
0	Biscuit	a	B	1
1	Chips	b	B	3

	D	A	variable	value
2	Banana	c	B	5
3	hard case	d	B	6
4	Biscuit	a	C	2
5	Chips	b	C	4
6	Banana	c	C	6
7	hard case	d	C	8

Joins

A join connects two dataframes (or SQL data tables) together based on matching values of keys (identifiers) in the two data frames or tables. You may have seen this in R, and we will see it again in SQL.

Joins are done on two dataframes (or tables) at a time, the first is called the "left" table and the second is called the "right" table and several different forms of joins exist.

Joins are done on Pandas data frames are done using the merge function

You can specify the type of join desired, inner, outer, left, right etc

<https://pandas.pydata.org/docs/reference/api/pandas.merge.html>

Joins are also common when working with SQL, so we will see the join concept again later when we discuss SQL

In []:

In []:

```
In [51]: df1 = pd.DataFrame({'lkey': ['foo', 'bar', 'baz', 'foo', "bix"], 'value': [1, 2, 3, 5, 11]})
df2 = pd.DataFrame({'rkey': ['foo', 'bar', 'baz', 'foo'], 'value': [5, 6, 7, 8]})
```

In [52]:

df1

Out[52]:

	lkey	value
0	foo	1
1	bar	2
2	baz	3
3	foo	5
4	bix	11

In [53]:

```
df2
```

Out[53]:

	rkey	value
0	foo	5
1	bar	6
2	baz	7
3	foo	8

In [54]:

```
# this is an inner join of the left frame df1 and the right frame df2
# notice that "bix" was dropped
df1.merge(df2, left_on='lkey', right_on='rkey')
```

Out[54]:

	lkey	value_x	rkey	value_y
0	foo	1	foo	5
1	foo	1	foo	8
2	foo	5	foo	5
3	foo	5	foo	8
4	bar	2	bar	6
5	baz	3	baz	7

In [55]:

```
# here is a left join
# what happens to "bix"?
df1.merge(df2, how="left", left_on='lkey', right_on='rkey')
```

Out[55]:

	lkey	value_x	rkey	value_y
0	foo	1	foo	5.0
1	foo	1	foo	8.0
2	bar	2	bar	6.0
3	baz	3	baz	7.0
4	foo	5	foo	5.0
5	foo	5	foo	8.0
6	bix	11	NaN	NaN

In []:

Bix remained because bix **is in** the left one **and** it was a left join **not** one where Left join showcases all of the variables **in** the left category.

In []:

In []:

In []:

In []:

In []:

In []: